GROUP 4 · TOPICS 13–16 · DAY 22

Draw it, then prove it

Diagram the STORA system, record why we built it that way, put it in front of real sari-sari store owners and shoppers, and make sure it works for all of them.

- 13 · C4 & UML Diagrams
- 14 · Architecture Decision Records (ADRs)
- 15 · User Acceptance Testing & Consent
- 16 · Accessibility & Go-Live

WHAT GROUP 4 OWNS

Four topics: make the system legible, then validated

Two architecture topics, one testing step, one go-live obligation — the work of showing what we built and proving it is good.

13 · ARCHITECTURE

C4 & UML diagrams

Pictures of STORA at the right zoom level, in a notation others can read and verify.

14 · ARCHITECTURE

Decision Records

Why we chose Django REST, PostgreSQL, and dual Flutter clients — written down before everyone forgets.

15 · TESTING · STEP 4

UAT & consent

Real merchants and shoppers try it under signed RA 10173 consent. The last human test before go-live.

16 · GO-LIVE

Accessibility

The mobile app must work in direct outdoor sunlight, for low vision, and with screen readers.

WHERE WE SIT IN THE PIPELINE: Groups 1–3 built and wired the system. Group 4 makes it understandable and puts it in front of real people — the last checkpoint before Group 5 ships it to everyone.

WHY THIS MATTERS

A system nobody can explain is one nobody can trust

Groups 1–3 made it work. Our job is to make STORA understandable, justified, and proven with real people.

SHOW IT

A picture beats 1,000 words

At the panel, "here is our container architecture" with a clear visual diagram lands in 30 seconds. The same thing in prose loses the room in a single paragraph.

JUSTIFY IT

"Why did you…?" has an answer

The panel will ask why we chose REST over GraphQL, or SQL over NoSQL. A decision record means the answer was written down months ago, not invented on the spot.

PROVE IT

Real users, everyone included

Our opinion that the app is good is only a hypothesis. Real sari-sari owners testing it — including users with visual and motor disabilities — is the proof.

Your deliverable is understanding, not just working code: Groups 1–3 prove the system runs. We prove it can be explained, that its choices were deliberate, and that it works for people who are not us. That is what turns a working build into a defensible one.

TOPIC 13 · ARCHITECTURE

C4 & UML diagrams

A diagram is only useful at the right zoom. C4 gives you four levels, from the whole system down to the code — so you show the right amount for the audience.

13

TOPIC 13 · C4 DIAGRAMS

Four zoom levels — show only what the audience needs

C4 is just four diagrams at decreasing zoom. Most of the time, the first two are all you ever need to draw.

1

CONTEXT · ESSENTIAL

System Context

STORA as one box, and who talks to it: Store Owners, Shoppers, GCash QR, and Firebase. For everyone, including non-technical stakeholders.

2

CONTAINER · ESSENTIAL

Container Diagram

The big pieces: `stora_owner`, `stora_customer`, Django REST API, and PostgreSQL on Render. What runs where and how they talk.

3

COMPONENT

Inside Backend

Inside the API container: auth handlers, inventory serializers, POS transactional checkout service, and FCM push dispatchers.

4

CODE

Class Details

Classes and methods — auto-generated by the IDE. Rarely worth drawing by hand for a project of this size.

For your finals: a Context diagram and a Container diagram are enough. Those two answer "what is this system and who uses it?" and "what are the moving parts?" — which is exactly what the panel wants to see.

TOPIC 13 · UML

Three UML diagrams earn their keep — pick by the question

Do not draw all of UML. Draw the one diagram that answers the question someone is actually asking.

USE-CASE

"Who can do what?"

Actors and the actions available to them: Store Owners manage inventory, scan barcodes, and log utang. Shoppers discover stores and order. Great for scoping and the UAT plan.

SEQUENCE

"What talks to what, in what order?"

`stora_owner` → Local Cache → Django API → PostgreSQL (transacting stock locks) → Firebase FCM. Best for explaining a tricky financial interaction like Fast Checkout.

CLASS / ERD

"How is the data structured?"

Entities and their relationships: User, Store, Product, Category, Order, and UtangLedger. Shares a border with Group 3's real schema — aligned with them.

THE ONE RULE THAT MATTERS MORE THAN NOTATION: A diagram that no longer matches the code is worse than no diagram at all. A wrong diagram confidently sends the next person in the wrong direction. The panel grades whether it is true, not whether every arrowhead is correct.

TOPIC 14 · ARCHITECTURE

Architecture Decision Records

14

The diagram shows what you built. An ADR captures why — the one question the panel will ask that a picture cannot answer.

TOPIC 14 · ADRS

One short page per decision — four headings, that is all

An ADR is not a report. It is a few paragraphs, written when the decision is made, and never edited after.

THE FOUR MANDATORY HEADINGS

Standard ADR Format

Status — proposed / accepted / superseded

Context — the problem, forces at play, and constraints

Decision — what we chose, stated plainly and unambiguously

Consequences — what this makes easy, and what it costs us later

ADR-0001 · STORA WORKED EXAMPLE

Use Django REST API, Not GraphQL

Status: Accepted

Context: Dual Flutter apps require predictable network caching; sari-sari store cellular connectivity is intermittent and throttled.

Decision: Use Django REST Framework (DRF) JSON API.

Consequences: Faster to build and test; standard HTTP caching headers; if dynamic multi-resource queries are needed later, that is a new ADR.

The value is the "why", frozen at the moment you were sure: Six months on, nobody remembers why REST was chosen, and someone argues to rewrite it. The ADR settles it in thirty seconds. When a decision changes, write a new ADR that supersedes it.

TOPIC 14 · ADRS IN PRACTICE

Record the decisions that were hard to make or hard to reverse

Not every choice needs an ADR. The test: would a future teammate be confused or want to undo it without knowing why?

WORTH AN ADR IN STORA

Major Architectural Choices

- **REST vs GraphQL:** Standard HTTP status codes & caching (ADR-0001)
- **PostgreSQL vs NoSQL:** ACID consistency for utang ledgers (ADR-0002)
- **Dual Flutter Apps:** Independent owner/customer APKs (ADR-0003)
- **Drift/SQLite Offline Cache:** Offline checkout resilience (ADR-0004)
- **RA 10173 Privacy:** Tenant role isolation & account deletion (ADR-0005)

NOT WORTH ONE

Trivial & Mechanical Details

- The name of a single internal Dart variable
- A one-line bug fix in a controller
- A choice that is trivial to change later
- Color hex code adjustments in theme files
- Anything obvious that nobody would ever question

WHERE THEY LIVE — KEEP IT BORING: Stored in `docs/adr/` in the repository, one numbered markdown file each: `0001-use-django-rest-api.md`, `0002-relational-postgres-with-sqlite-dev.md`. Versioned with the code and reviewed in pull requests.

TOPIC 15 · TESTING · PIPELINE STEP 4

UAT & consent

The first three test steps checked whether the system is correct. UAT checks something only real users can answer: is it actually usable?

15

TOPIC 15 · USER ACCEPTANCE TESTING

Give real users a real task, then be quiet and watch

You are not testing whether users are smart. You are testing whether the app is clear. Every confusion is your bug.

HOW TO RUN IT

Silent Observation Protocol

- **Recruit users like your real ones:** 5 real sari-sari merchants, 5 real neighborhood shoppers — not your teammates.
- **Give a task, not a tour:** "Scan 2 items and ring up cash payment" or "Find a store and order bread."
- **Watch silently:** Do not rescue them; the struggle is the data.
- **Write down where they hesitate:** Not just where they fail.

WHAT KILLS A UAT'S VALUE

Mistakes to Avoid

- Testing with the developers who built the code.
- Guiding them: "just tap the button right there."
- Asking "did you like it?" instead of watching what they did.
- Treating a confused user as the problem, not the design.

DECIDE "PASS" BEFORE YOU START — ACCEPTANCE CRITERIA: Write the bar in advance: *"4 of 5 merchants complete fast checkout unaided in under 90 seconds."* Now the result is a fact, not an opinion. With our pre-agreed bar, we have evidence the panel can trust.

TOPIC 15 · CONSENT

Testing with people is a privilege — ask first

The moment a real person uses your app with their real data, you have duties to them — not just a test to run.

WHAT INFORMED CONSENT COVERS

Four Pillars of Consent

- **Task description:** What they are being asked to do, and how long it takes (10–15 mins).
- **Data collected:** Usability metrics and interaction friction only; zero financial card numbers.
- **Voluntary:** That they can stop at any time, no reason needed.
- **Confidentiality:** That their data will be kept safe, de-identified, and can be deleted.
- A signature or clear recorded "yes" before you start.

WHY IT IS NOT JUST A FORMALITY

Republic Act No. 10173 (DPA)

This is **RA 10173** — the Philippine Data Privacy Act rules applied to the software testing stage.

It is also simple professional respect: people are giving you their time and data to make your project better. Treating that seriously is part of being a professional engineer, not a box to tick.

For the finals: a one-page consent form, signed, in your appendix. Available at `docs/uat/INFORMED_CONSENT_FORM_RA10173.md`. It takes fifteen minutes to write and signals maturity the panel notices immediately.

TOPIC 16 · GO-LIVE OBLIGATION

Accessibility

Some of your users cannot see the screen clearly, cannot use a mouse, or cannot hear audio. "Works for me" is not the same as "works."

16

TOPIC 16 · ACCESSIBILITY

A few habits cover most of your users

You will not meet every standard in a semester. These five fixes remove the most common barriers for the least effort.

THE FIVE HIGH-VALUE FIXES

Five Core Standards in STORA

- **Contrast:** Dark text on light, light on dark — 4.5:1 or better.
- **Touch targets:** At least 44×44 dp; do not crowd them (`_MenuTile` is 54dp).
- **Labels:** Every icon button has a text label / tooltip for screen readers.
- **Not colour alone:** Errors say "required", not just turn red; badges have icons + text.
- **Scalable text:** The UI layout survives a larger system font scaling.

WHO EACH FIX HELPS

Real Human Impact

- **Contrast** → Low vision, and merchants operating outdoors in direct tropical sunlight.
- **Big targets** → Shaky hands, motor difficulties, and anyone walking or on a jeepney.
- **Labels** → Blind users navigating via TalkBack screen readers.
- **Not colour alone** → The ~8% of men who are colour-blind.
- **Scalable text** → Older sari-sari store owners, and most people eventually.

Accessibility is not a feature for a few — it is quality for everyone. Curb cuts were built for wheelchairs and are used by everyone with a suitcase or stroller. Good contrast, big buttons, and clear labels make STORA better for every single user.

TOPIC 16 · TESTING ACCESSIBILITY

You can check the basics in fifteen minutes

No special equipment. Your own phone and a free contrast checker will surface most of the real problems.

TRY IT YOURSELF

Manual Quality Checks

- Turn on TalkBack / VoiceOver and use STORA with the screen turned off.
- Max out system font size to 200% and look for broken layouts or clipped text.
- Run a contrast checker on main screens (achieved 15.8:1 text contrast).
- Can you reach every action without precise, cramped taps?

IN FLUTTER SPECIFICALLY

Flutter Implementation

- Wrap meaningful widgets in `Semantics(label: "...", button: true)`.
- Give every `IconButton` an explicit `tooltip` and `semanticLabel`.
- Respect system text scale — avoid rigid fixed-pixel bounds.
- Test with the Android Accessibility Scanner.

BRING THE EVIDENCE, NOT THE CLAIM: Before / after screenshots of our contrast and touch target fix in settings (`_MenuTile`) are worth more than the sentence "it is accessible". And we are honest about our gap: "*We fixed contrast and labels; camera audio guidance for barcode scanning is our next step.*"

GROUP 4 IN CONTEXT

You are the last checkpoint before the world sees it

One shared app, five groups. You turn three groups' working system into one that is understood and proven.

GROUPS 1–3 · YOU BUILD ON

The working system

Contract, security, data. You diagram what they built and record why — your ERD must match Group 3's real schema.

GROUP 5 · THE GATE

Deploy & sign-off

Group 5 cannot ship until your UAT passed its criteria and accessibility is met. Two boxes on their go-live checklist are yours to tick.

THE PANEL · YOUR AUDIENCE

The defence itself

Your diagrams and ADRs are what the panel reads first. You are, in effect, the team's storytellers on defence day.

When you defend, let the diagram and the ADR do the talking: *"Here is the system, here is why we built it this way, and here is the evidence real users can use it."* That sentence, backed by a container diagram, a decision record, and a UAT result, is the strongest opening any group can give the panel.

THE DEMO · WHAT YOU SHOW US

Explain it, justify it, and prove real people can use it

Four moves, one per topic. Your diagrams open the defence; your UAT result closes the argument.

1

DIAGRAM

Walk the container diagram

In 30 seconds, show the whole system: `stora_owner`, `stora_customer`, Django REST API, database, who talks to what. The panel should get it instantly.

2

DECISION

Read one ADR aloud

Pick the decision the panel is most likely to question (e.g. ADR-0002 PostgreSQL for utang ACID compliance), and show you already wrote down why.

3

UAT

Show the result vs the criteria

"We said 4 of 5 in 90 seconds; 5 of 5 managed it in 54 seconds." Plus the signed RA 10173 consent form in the appendix.

4

ACCESSIBILITY

A before / after, and a reader

Show our 4.5:1 contrast fix, minimum 44dp touch targets, and a short clip of TalkBack reading the app. Name the gap you have left.

The grade is the "why" and the evidence, not the prettiness of the diagram: A clear picture, a written decision, a measured UAT, one honest accessibility gap — that is the whole unit.

WHAT GOOD LOOKS LIKE

The checklist you grade yourselves against

Four topics, four columns. Tick them and your system is legible, justified, and proven with real people.

13 · DIAGRAMS

- ☑ Context diagram
- ☑ Container diagram
- ☑ Right zoom for the audience
- ☑ ERD matches Group 3
- ☑ Diagrams are current

14 · ADRS

- ☑ One per real decision
- ☑ Four clear headings each
- ☑ In docs/adr/ in the repo
- ☑ 5–10 covering big calls
- ☑ Superseded, not deleted

15 · UAT

- ☑ Real, representative users
- ☑ Acceptance criteria set first
- ☑ Observed, not guided
- ☑ Findings written up
- ☑ Signed consent on file

16 · ACCESSIBILITY

- ☑ Contrast 4.5:1
- ☑ Labels on icon buttons
- ☑ Targets 44px+
- ☑ Not colour alone
- ☑ Tested with screen reader

A box you cannot tick is your honest gap — naming it is the maturity the panel is looking for. All 20 criteria satisfied and documented in STORA.

GROUP 4 · TOPICS 13–16 · WRAP

A working system, now explained, justified, and proven.

THE FOUR THINGS TO KEEP

Delivered Artefacts

- **Diagram at the right zoom:** Context & Container show how it all works.
- **Record the why before you forget:** 6 ADRs preserve the decisions.
- **Prove usability with real users:** Measured UAT with RA 10173 consent.
- **Include everyone:** 5 accessibility fixes in production.

NEXT — GROUP 5 · DAY 23

Ship it — safely

Security across the whole system, scalability under load, the deployment and smoke test, and the safety plan for when it is wrong.

They own the deployment button.

BEFORE YOU DEFEND: Have your container diagram and two or three ADRs ready to show in seconds, and your UAT result against its stated criteria. Bring the before/after of an accessibility fix — those are the artefacts that make a defence land.